

Instrukcja: produkcyjny workflow CloudKit i APNs dla PavbotViewer

Data: 2026-07-06 Zakres: **com.paweltanski.pavbotviewer**, Team ID **SP774TZZU8**, CloudKit Production

Cel

Ta instrukcja pokazuje, jak działa aktualny produkcyjny system PavbotViewer:

- Codex publikuje artefakty tematu i odświeża **public/pavbot-manifest.json**.
- GitHub **origin/main** jest źródłem prawdy dla manifestu i plików.
- CloudKit dostaje jeden rekord **Briefing** dla aktywnego runu.
- iOS tworzy subskrypcję CloudKit i przez APNs dostaje widoczny alert oraz sygnał odświeżenia.
- Push Notifications Console służy do ręcznych testów APNs, walidacji JWT i diagnostyki dostarczenia.

Źródła

- Apple: [Testing notifications using the Push Notification Console](#)
- Apple: [Sending notification requests to APNs](#)
- Apple: [Generating a remote notification](#)
- Lokalnie: **docs/CLOUDKIT_MIGRATION.md**
- Lokalnie: **docs/how-to-use.md**

Model produkcyjny

Produkcja nie polega na ręcznym wysyłaniu każdego powiadomienia z APNs Console. Normalna ścieżka wygląda tak:

```
Codex run -> topic artifacts -> public/pavbot-manifest.json -> origin/main  
-> CloudKit Briefing(status=ready) -> CloudKit Subscription -> APNs -> iOS
```

APNs Console jest narzędziem testowym. Używaj go do ręcznego smoke testu, sprawdzenia JWT, device tokenu i logów dostarczenia.

Publikacja runu

Przed publikacją upewnij się, że lokalny **cktool** ma ważny token:

```
xcrun cktool save-token
```

Następnie publikuj aktywny temat:

```
export PAVBOT_CLOUDKIT_CONTAINER_ID=iCloud.com.paweltanski.pavbotviewer  
export PAVBOT_CLOUDKIT_ENVIRONMENT=production  
export PAVBOT_CLOUDKIT_TEAM_ID=SP774TZZU8
```

```
scripts/pavbot_commit_and_push_outputs.sh --isolated research/<topic>
```

Kolejność po naprawie systemu:

```
prepare -> validate -> manifest -> CloudKit preflight -> commit/push -> remote verify -> CloudKit  
publish -> CloudKit verify
```

Najważniejsza zmiana: CloudKit preflight działa przed commitem i pushem. Jeśli token CloudKit wygasł, skrypt zatrzyma się zanim zmieni **origin/main**.

Naprawa po częściowej publikacji

Jeśli manifest i artefakty są już na **origin/main**, ale CloudKit nie dostał rekordu, odśwież token:

```
xcrun cktool save-token
```

Potem uruchom tryb naprawczy:

```
export PAVBOT_CLOUDKIT_CONTAINER_ID=iCloud.com.paweltanski.pavbotviewer  
export PAVBOT_CLOUDKIT_ENVIRONMENT=production  
export PAVBOT_CLOUDKIT_TEAM_ID=SP774TZZU8  
scripts/pavbot_commit_and_push_outputs.sh --cloudkit-only research/<topic>
```

Ten tryb:

- sprawdza **origin/main**,
- synchronizuje lokalny manifest z wersją zdalną,
- publikuje rekord **Briefing**,
- weryfikuje rekord w CloudKit,
- nie tworzy nowego commita.

Ręczny test w Push Notifications Console

Otwórz:

```
https://icloud.developer.apple.com/dashboard/notifications/teams/SP774TZZU8/app/com.paweltanski.pavbotviewer/tools/validateJwt
```

1. JWT Generator

W narzędziach konsoli wybierz JWT Generator:

- wgraj prywatny plik **.p8**,
- wybierz właściwy Key ID,
- wygeneruj JWT,
- nie zapisuj JWT w repozytorium ani w dokumentach.

Apple informuje, że generator działa w przeglądarce, ale nadal traktuj **.p8** i wygenerowany token jako sekrety produkcyjne.

2. JWT Validator

Wklej wygenerowany JWT do pola **Encoded Token** i zwaliduj:

- podpis,
- Team ID,
- czas ważności,
- zgodność z kluczem.

Jeśli walidator mówi o wygaśnięciu lub błędnym tokenie, wygeneruj nowy JWT.

3. Device Token Validator

Wklej produkcyjny device token z aplikacji. Nie wpisuj go do repozytorium. Sprawdź, czy token należy do:

```
com.paweltanski.pavbotviewer
```

oraz środowiska:

```
Production
```

4. Create Notification

W formularzu wysyłki ustaw:

- Name: **Pavbot production APNs test**
- Environment: **Production**
- Device token: produkcyjny token urządzenia
- **apns-topic: com.paweltanski.pavbotviewer**
- **apns-push-type: alert**
- Expiration: **Attempt delivery once**
- Priority: **High (10)**
- Payload: włącz **JSON View**

Payload testowy:

```
{
  "aps": {
    "alert": {
      "title": "Pavbot",
      "subtitle": "Test produkcyjny",
      "body": "Nowe dane: <BRIEFING_TITLE>"
    },
    "sound": "default",
    "content-available": 1
  },
  "briefingId": "<TOPIC>:<STAMP>",
  "category": "<TOPIC>",
  "title": "<BRIEFING_TITLE>",
  "summary": "<BRIEFING_SUMMARY>",
  "manifestUrl":
    "https://raw.githubusercontent.com/19paoletto10-hub/pavbot-public-data/main/public/pavbot-manifest.j
    son"
```

```
}
```

Ten payload jest do ręcznego smoke testu. Produkcyjna ścieżka w aplikacji i tak powinna opierać się na rekordzie CloudKit **Briefing**.

Checklist dla udanego runu

- **scripts/pavbot_commit_and_push_outputs.sh --isolated research/<topic>** kończy się kodem 0.
- Zdalny **public/pavbot-manifest.json** na **origin/main** zawiera nowy run.
- Pliki z manifestu istnieją na **origin/main**.
- CloudKit zawiera **Briefing** z **status = "ready"**.
- iOS dostaje alert APNs z subskrypcji CloudKit.
- **scripts/verify-research-workspace.sh** kończy się bez błędów.

Bezpieczeństwo

Nie zapisuj w repozytorium:

- device tokenów,
- plików **.p8**,
- JWT,
- tokenów **cktool**,
- prywatnych screenshotów z danymi urządzenia.

W dokumentacji używaj placeholderów takich jak **<DEVICE_TOKEN>**, **<KEY_ID>**, **<TOPIC>** i **<STAMP>**.